# Representing Knowledge through Relational Databases- Part 3

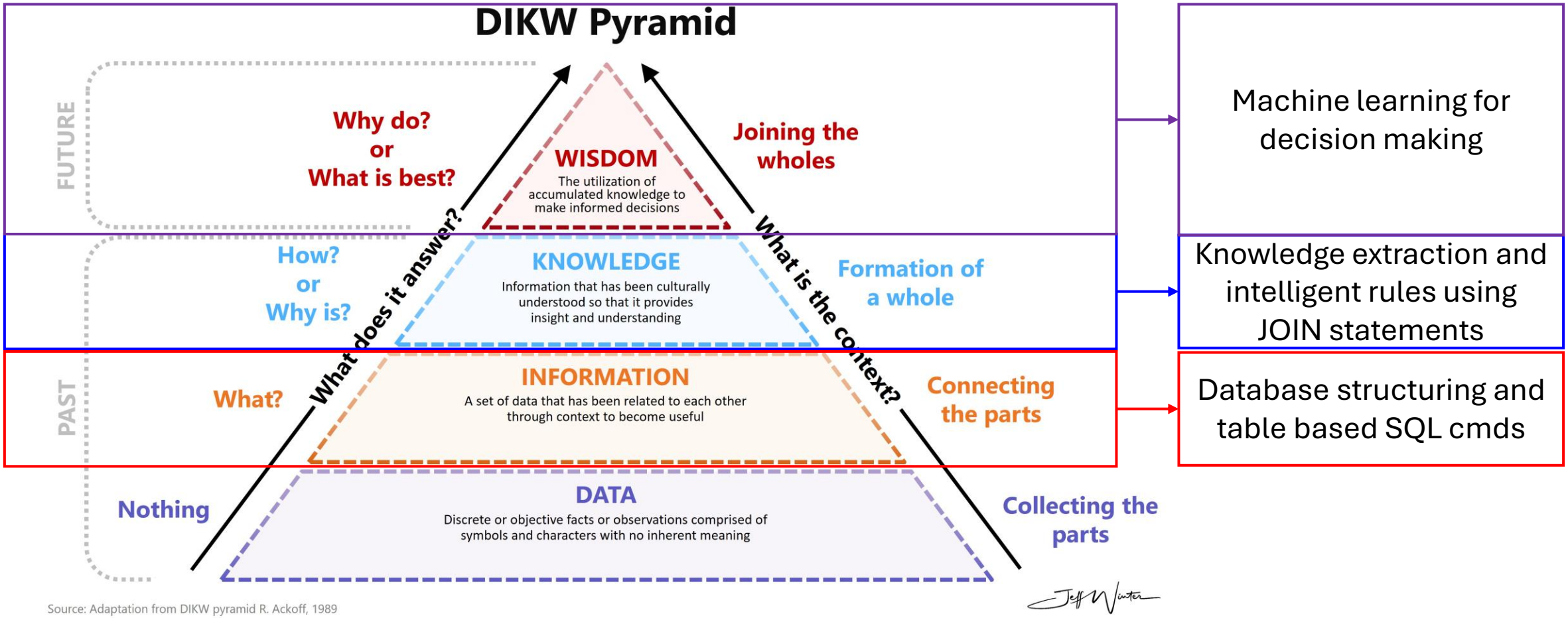
**Moving from knowledge onto insights**

John Oyekan

# Schedule for the next 3-4 weeks

- **Developing the information layer (Week 4)**
  - Entity Relationship diagram and table querying for information
- **Developing the knowledge layer (Week 5)**
  - JOIN SQL queries for forming of a whole and intelligent rules
- **Developing the wisdom layer (Week 6)**
  - Unified Modelling Language and applying machine learning
- **Data visualisation to support further in the development of the wisdom layer (Week 7)**
  - Machine learning and types of data visualisation

# Data, Information, Knowledge and Wisdom Pyramid




Source: Adaptation from DIKW pyramid R. Ackoff, 1989

# Recap: Importance of database structure

- The design of the database at the start is very important.
- It can make things easy or very difficult later on.
- So it is important you get it right first time.

**Important tips:**

- Knowing the purpose of the database is important.
- Understanding the domain is important.
- Understanding the services that the domain might want to conduct on datasets is important.
- This are often captured in a requirements document to be signed off by the clients!

# Importance of database structure

- Once structure is determined and designed well.
- Then services, functions, intelligence and other high level behaviour can then be added onto the foundation of the database.
- A strong foundation would make adding of these high level functions easier as we will see.
- **But how do we do that in a structured way?**

# Entity Relation Diagram Disadvantages

- **Complexity and Size:** As database size increases, ERDs can become overwhelmed with too many entities and relationships, making them hard to read and manage.
- **Limited Representation:** They often fail to accurately represent complex, non-binary relationships (e.g., ternary relationships), requiring them to be broken down, which reduces clarity.
- **Lack of Standardization:** There is no universal standard for notation, which can cause confusion and misinterpretation among different designers.
- **Rigidity in Design:** Changes to the database structure during later phases are difficult and time-consuming to reflect in the diagram.
- **Difficulty with Non-Relational Data:** ERDs are designed for relational databases and are ineffective for NoSQL, graph, or unstructured data.

# Developing the knowledge layer

- **Lack of Data Manipulation:** They cannot represent data manipulation processes like INSERT, UPDATE, or DELETE.
- How do we know which intelligence functions to apply for data processing?
- UML is the answer!
- This does not mean that ER is useless!

# Developing the knowledge layer

- **Use ER to quickly identify Entities and Relationships.**
- **Enrich your ER diagram by expanding it into UML!**
- **So what is UML?**

# UML Modelling Language

- Unified Modeling Language (UML) is a standardized specification language for object oriented modelling.

- UML is a general-purpose modelling language that includes a graphical notation used to create an abstract model of a system, referred to as a UML model.

# Classes

- A class represent real-world things or conceptual things, including the thing's characteristics (its attributes or properties) and the things it can do (its behaviours or methods).

- Example: modelling civilizations

# Attributes and behaviours

- A **class has attributes**
- Romans' characteristics:
  - Eyes colour
  - Hair colour
  - Height
  - Weight

- A **class has behaviours/methods**
- Romans' abilities:
  - To conquer
  - To watch gladiators
  - To eat

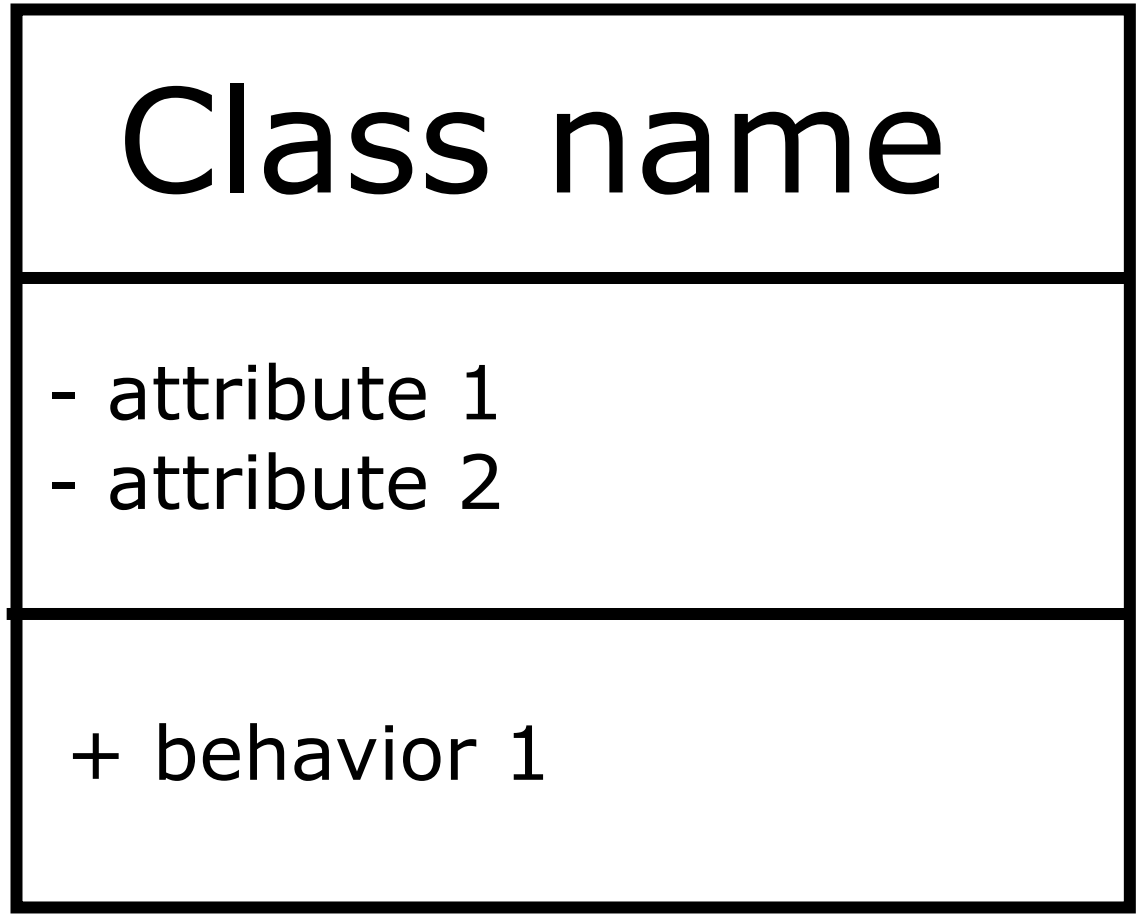
# Objects

- A particular instance (example) of a class.

**Class: Roman**

**Instance: Julius Cesar**

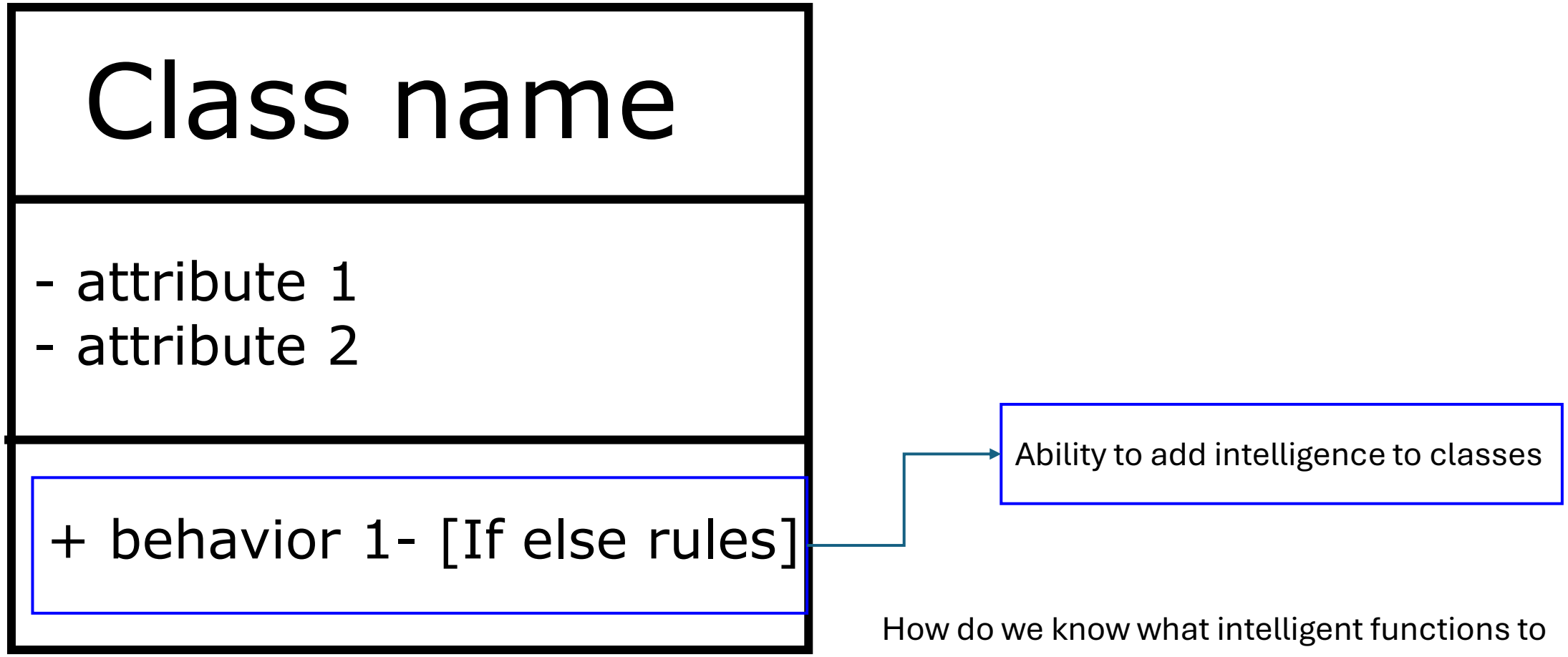
# UML Class representation



# Use case: Vikings

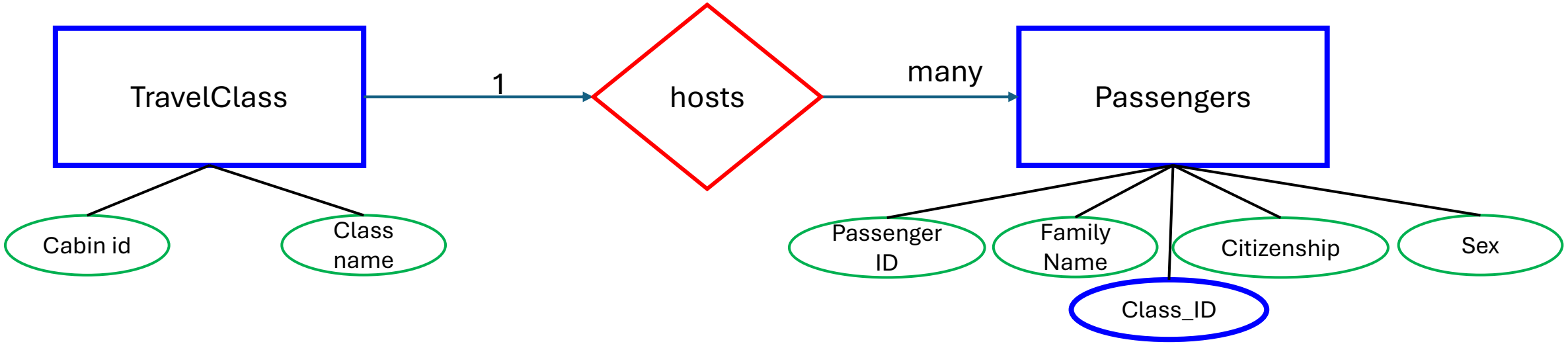
What are the attributes and behaviours/functions that we might want to create for this domain?

# UML Class representation

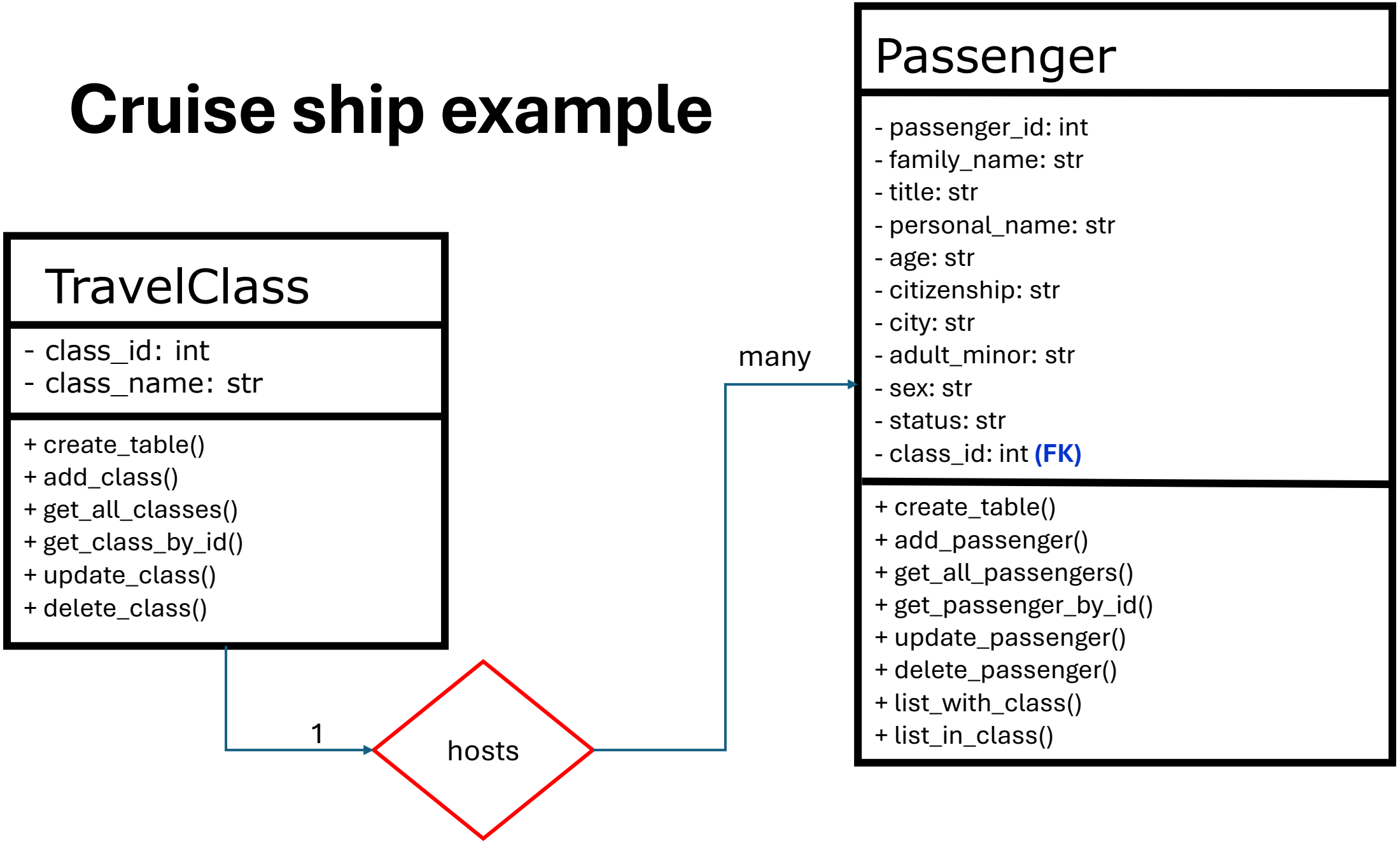



How do we know what intelligent functions to add to our classes?

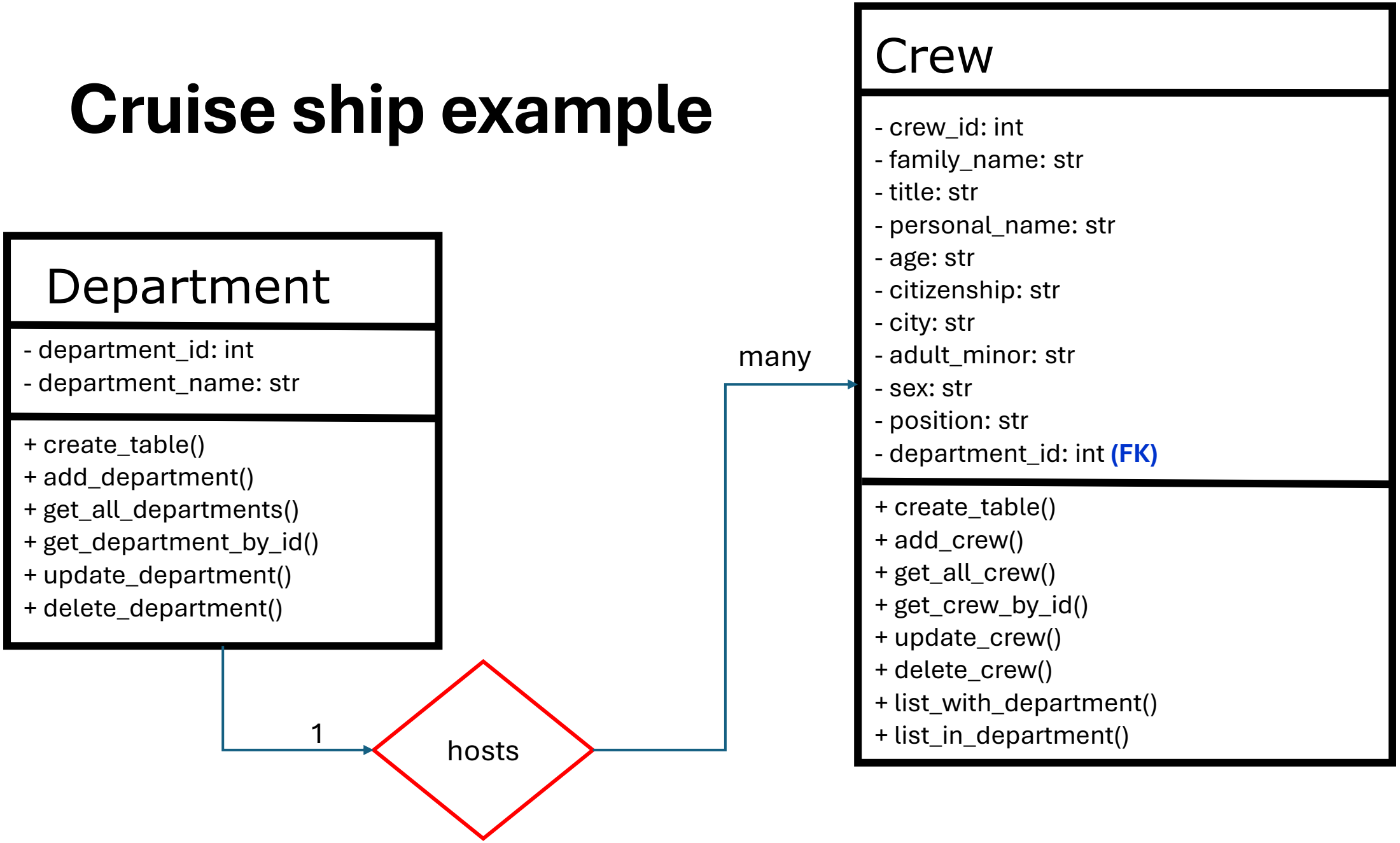
# SQL commands to use for one to many



# Cruise ship example



# Cruise ship example



# Adding CRUD functions

- When designing **classes that map to database tables**, a good rule of thumb is that each entity should support the core **CRUD operations.**

- CRUD means
  - **Create**
  - **Read**
  - **Update**
  - **Delete**

**Crew**

- crew_id: int
- family_name: str
- title: str
- personal_name: str
- age: str
- citizenship: str
- city: str
- adult_minor: str
- sex: str
- position: str
- department_id: int **(FK)**

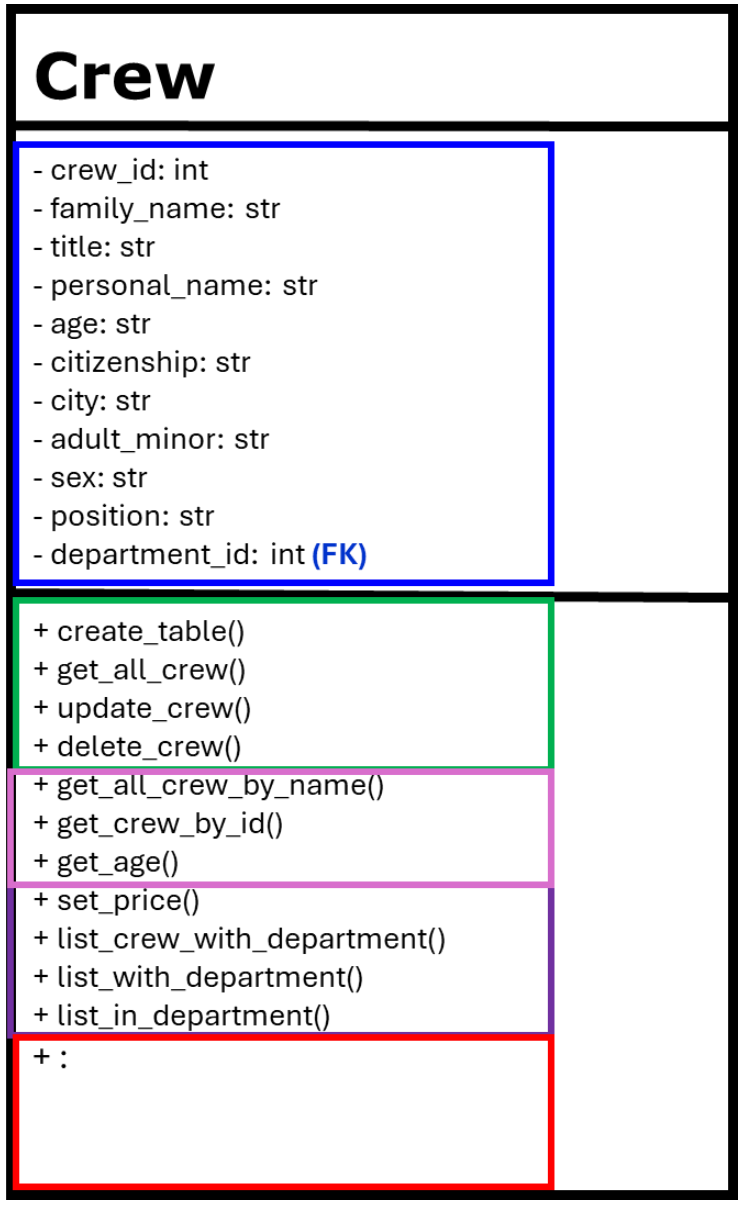
+ create_table()
+ get_all_crew()
+ update_crew()
+ delete_crew()
+ get_all_crew_by_name()
+ get_crew_by_id()
+ get_age()
+ set_price()
+ list_crew_with_department()
+ list_with_department()
+ list_in_department()
+ :

# Adding information extraction via functions

These functions focus on table level SQL queries.

**Useful for:**

- Diagnosis, data cleaning activities
- Visualisation of data distribution and exploration of data trends.



# Adding knowledge derivation and extraction via functions

- This is where you add **relationship-related operations** across tables.

- The use of JOIN SQL queries is used here
- The FOREIGN KEYS become very useful here.

**Crew**

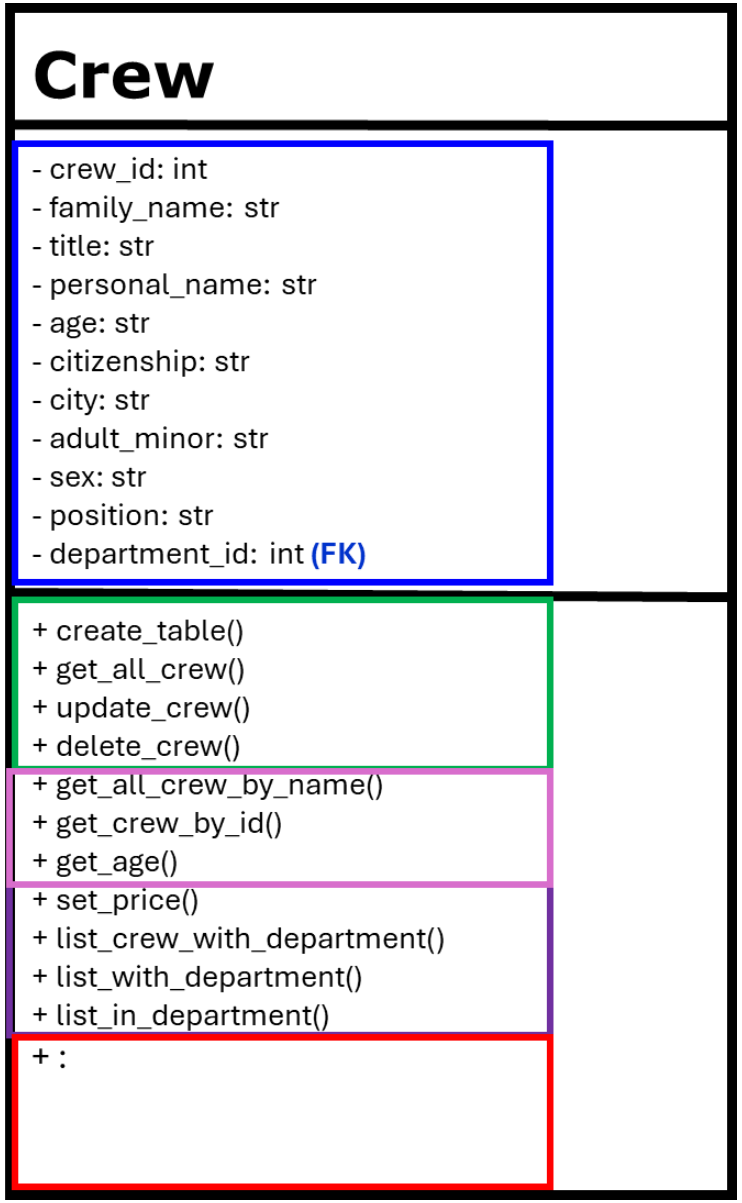
- crew_id: int
- family_name: str
- title: str
- personal_name: str
- age: str
- citizenship: str
- city: str
- adult_minor: str
- sex: str
- position: str
- department_id: int **(FK)**

+ create_table()
+ get_all_crew()
+ update_crew()
+ delete_crew()
+ get_all_crew_by_name()
+ get_crew_by_id()
+ get_age()
+ set_price()
+ list_crew_with_department()
+ list_with_department()
+ list_in_department()
+ :

# Adding wisdom/insight code to UML

Machine learning algorithms that need to be trained and used for prediction fit in here including:

- Decision Trees
- Neural Networks
- etc



# Cruise ship example: Behaviour + Rules (Add more functions)

## Crew

- crew_id: int
- family_name: str
- title: str
- personal_name: str
- age: str
- citizenship: str
- city: str
- adult_minor: str
- sex: str
- position: str
- department_id: int **(FK)**

+ create_table()
+ get_all_crew()
+ update_crew()
+ delete_crew()
+ get_all_crew_by_name()
+ get_crew_by_id()
+ get_age()
+ set_price()
+ list_crew_with_department()
+ list_with_department()
+ list_in_department()
+ :

Table structure

Defines intelligence and behaviours of the entity/class to **derive or apply knowledge.** **Knowledge** = pulling information from multiple tables together using JOIN.

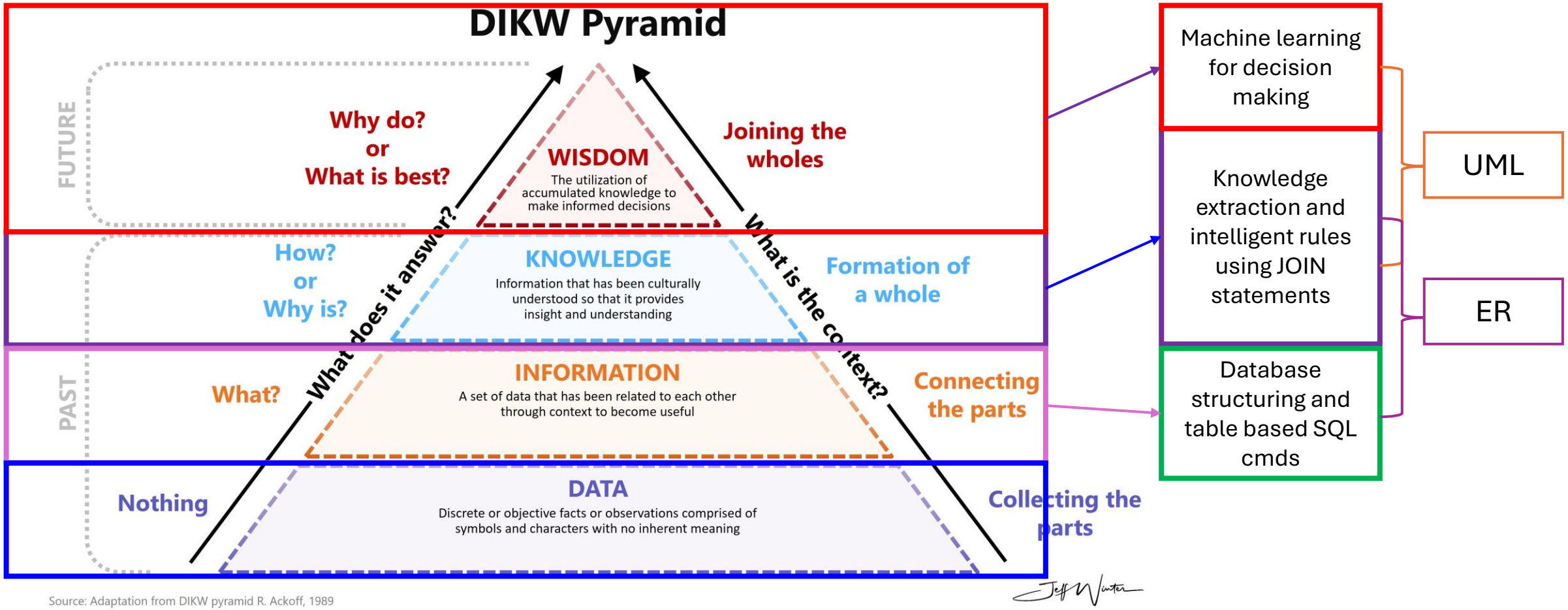
CRUD type operations

Information querying (get functions). **Information** = intelligently querying single tables.

set_price (float cabin_price): float
if passenger.age >= 65:
apply_discount()
apply_discount (int get_age): bool

Define **"wisdom"** functions for the entity/class. **Wisdom** = Combine intelligent querying with Machine Learning models

# Data, Information, Knowledge and Wisdom Pyramid




Source: Adaptation from DIKW pyramid R. Ackoff, 1989